

Can AI Be Small? A Complete, Explainable, Multi-Objective Toolkit in 444 Lines

TIM MENZIES, North Carolina State University, USA

Much recent commentary asserts that effective AI requires large models, large data, and large infrastructure. We offer a counter-example, at least for tabular software-engineering optimization tasks where independent attributes are cheap to read but dependent goals are expensive to measure. This paper presents, in its entirety, `eZR.py`: 444 lines of dependency-free Python implementing incremental column summaries, multi-objective distance, active learning, Naive Bayes classification, classification and regression trees, a confusion-matrix scorer, and the non-parametric statistics needed to check all of the above. On the 120+ tasks of the MOOT repository, this code's active learner finds near-optimal rows after labelling around fifty examples, matching heavyweight optimizers that run orders of magnitude slower. Unusually, the paper's format matches its thesis: every line of the toolkit appears here, one conceptual unit per page, with commentary alongside. We argue that when a complete system fits in a paper's figures, reading code becomes a research method: the simplifications reported here were found not by adding machinery, but by deleting it.

1 INTRODUCTION

It is now routinely argued that humans no longer need to read or write code; that the era of hand-crafted software is ending; that AI is, by nature, big. Whatever the merits of those claims for perception or generation tasks, they are testable in at least one domain: tabular software-engineering optimization, where each row's independent attributes cost nothing to read, but each row's goals (runtimes, defect counts, build outcomes) cost a measurement. In that domain, this paper defends a simple thesis: *the AI that matters fits on a page*—or, more precisely, on eight of them.

The vehicle for that argument is `eZR.py`, a rewrite of our earlier EZR toolkit that emerged from a months-long exercise in deletion. Version by version (three complete rewrites, each verified output-identical on 120+ datasets before being trusted), classes collapsed into tuples and dicts, separate add/subtract/summarize routines collapsed into one update primitive, and separate learners collapsed into short questions asked of shared summaries. The result is 444 lines, zero dependencies, in which:

- a *Num* is a tuple (n, μ, m_2) and a *Sym* is a dict of counts (Listing 2);
- one primitive, `add` (whose `inc=-1` undoes), maintains both, so rows can move between summaries in constant time;
- active learning labels ~50 of thousands of rows, then a small tree ranks the rest (Listings 3, 5);
- Naive Bayes classification and its confusion-matrix scoring share the same columns (Listing 4);
- effect-size and distribution statistics—Cliff's delta, Kolmogorov–Smirnov, Cohen's d —check every comparison the toolkit makes about itself (Listing 6).

Our contribution is not any single algorithm; all are decades old. It is the demonstration, in full, that they share so much structure that a complete, statistically self-checking, explainable toolkit needs less code than many papers' related-work sections. Prior work reported this quantitatively: active learning on these summaries matches state-of-the-art optimizers such as SMAC while running orders of magnitude faster, and the trees it grows explain themselves in under ten attributes even when thousands are available. This paper adds the qualitative case: we show every line, because at this size we can.

The paper's format follows its content. Section 2 and onward present `ezr.py` one conceptual unit per page: the listing (left) with commentary (right). Line numbers are those of the released file (github.com/timm/ezr; `pip install ezr`); nothing is elided. Data comes from the MOOT repository of multi-objective SE tasks. We close with the threats and the invitation: before deploying a heavyweight pipeline, run a small baseline—if the heavyweight cannot beat these eight pages, it is technical debt.

2 THE CODE

Each of the following pages is one conceptual unit of `ezr.py`. Read the listings in order and you have read the entire system; read the right-hand columns and you have read its design rationale. The units are: configuration; structs; distance and acquisition; Bayes; trees; reports and statistics; evaluation and demos; and the sixteen-line harness that replaces a test framework.

Every setting lives once, in the docstring, as `-Key=default` plus a comment. One regex turns that text into the global the; `-help` prints the same text. Config, help and documentation cannot drift, because they are one artifact.

`atom` coerces strings once, at the border; everything downstream trusts its types. Its ordering is load-bearing: we tried a “faster” variant that returned only floats, and it was both slower (532 vs. 497 cpu-seconds over 120+ datasets—most cells are integers, so `int` succeeds first try either way) and wrong (settings such as `Few=128` became floats and crashed every slice that consumed them). A “richer” variant that stripped and tested for booleans on every cell cost 13% of total runtime. The shipped compromise recognizes booleans only on the rare path, after both numeric parses fail: measured cost, under 3%, in the noise.

The default snapshot supports a contract used throughout: demos may mutate the freely, and the runner restores pristine settings afterwards, so no demo can leak into the next. The CSV reader is five lines: no dtype inference, no chunked reads, no NA conversion—pandas offers all that, and this paper needs none of it. The \$MOOT expansion means per-machine data without per-machine edits.

```
#!/usr/bin/env python3 -B
"""
ezr.py: minimal XAI for multi-objective reasoning
(c) 2026 Tim Menzies <timme@ieee.org> MIT license
Options:
-P=2      minkowski coefficient
-Start=4   acquire: initial random labels
-Stop=50   acquire: total labelling budget
-Few=128   max train rows
-Leaf=4    tree: min rows in any leaf
-Check=5   holdout: top picks to label
-k=1       bayes: rare klass hack
-n=2       bayes: rare evidence hack
-Klass=$MOOT/classify/diabetes.csv classify demo data
-Repeats=30 klass: number of train/test splits
-Seed=1234567891 random number seed
-File=$MOOT/optimize/misc/auto93.csv
"""
# pylint: disable=bad-indentation,invalid-name
# pylint: disable=missing-function-docstring
# pylint: disable=multiple-statements,multiple-imports
# pylint: disable=unnecessary-lambda-assignment
# pylint: disable=inconsistent-return-statements
# pylint: disable=dangerous-default-value
# pylint: disable=broad-exception-caught
# pylint: disable=unidiomatic-typecheck
import os, random, re, sys, traceback
from math import exp, log, log2, pi, sqrt
from types import SimpleNamespace as o

def atom(s, bools={'True': True, 'False': False}):
    try: return int(s)
    except ValueError:
        try: return float(s)
        except ValueError:
            s = s.strip()
            return bools.get(s, s)

pat = r"(\w+)=(\S+)"
the = o(*(k: atom(v) for k,v in re.findall(pat, __doc__ or "")))
defaults = o(**vars(the))

def csv(file):
    file = file.replace("$MOOT", os.environ.get("MOOT")
    or os.path.expanduser("~/github/moot"), 1)
    with open(file, encoding="utf-8") as f:
        return [tuple(atom(x) for x in line.split(","))
                for line in f if line.strip()]
```

Listing 1. Configuration and input. The docstring is the single source of truth for all settings.

The whole substrate is two tiny types. A Num is a tuple (n, μ, m_2) —all that Welford’s algorithm needs to track a running mean and standard deviation, in one pass, without catastrophic cancellation. A Sym is a dict of counts. One primitive, add, updates either; called with `inc=-1` it *undoes*. That symmetry is what makes data movement cheap everywhere else: the tree’s cut sweeper slides values between summaries with one add each way, and the active learner demotes a row from its elite pool in constant time. There is no retraining step anywhere in this paper.

Two traps are worth recording. `adds` guards with `if it is None`, not `it or Num()`: an empty `Sym()` is falsy, and the shorter idiom silently replaces a fresh symbol counter with a number summary. And code testing “is this the first column?” must write `is not None`, because column index zero is falsy too.

`Tbl` routes columns by header convention: upper-case names are numeric; +/- suffixes mark goals to maximize or minimize; ! marks a class column; X means ignore. Decisions made once, at header time, propagate everywhere—no re-parsing of names later. One type test (`type(col) is Sym`) plus functions-as-values buys what class hierarchies buy, without the hierarchy; the pattern is the same one the Linux kernel uses with structs of function pointers, thirty years and no classes.

```

55  #-- structs -----
    Num = lambda: (0, 0, 0) # n, mu, m2: all Welford keeps
    Sym = dict

    def sd(col): return 0 if col[0] < 2 else sqrt(col[2]/(col[0]-1))

60  def add(col, v, inc=1): # new Num, or updated Sym; inc=-1 undoes
    if v == "?": return col
    if type(col) is Sym: col[v] = col.get(v, 0) + inc; return col
    n, mu, m2 = col
    n += inc
    mu = v - mu
    m2 += inc * d / max(1, n)
    return (n, mu, max(0, m2 + inc * d * (v - mu)))

70  def adds(lst, it=None): # accumulate a list into it
    if it is None: it = Num() # NB: "it or Num()" would
    for y in lst: it = add(it, y) # clobber an empty Sym()
    return it

75  def size(col):
    return sum(col.values()) if type(col) is Sym else col[0]

    def div(col): # Num: sd. Sym: entropy
    if type(col) is not Sym: return sd(col)
    n = sum(col.values())
    return -sum(v/n * log2(v/n) for v in col.values() if v>0)

    def Tbl(src):
    tbl = o(rows=[], cols={}, x=[], y={}, names=src[0], klass=None)
    for at, s in enumerate(tbl.names):
    if not s.endswith("X"):
    tbl.cols[at] = Num() if s[0].isupper() else Sym()
    if s[-1] == "!": tbl.klass = at
    elif s[-1] in "+-": tbl.y[at] = s[-1] == "+"
    else: tbl.x.append(at)
    for row in src[1:]: addRow(tbl, row)
    return tbl

    def clone(tbl, rows=[]): return Tbl([tbl.names] + rows)

95  def addRow(tbl, row=None, inc=1): # inc=-1 pops the last row
    if inc > 0: tbl.rows.append(row)
    else: row = tbl.rows.pop()
    for at in tbl.cols:
    tbl.cols[at] = add(tbl.cols[at], row[at], inc)
100  return row

```

Listing 2. Structs: Num, Sym, and the one update primitive that both builds and unbuilds them.

Numbers are normalized by a clamped z-score pushed through a logistic. The clamp (at ± 3 standard deviations) stops one outlier crushing everyone else onto a dot; the 1.7 in the logistic makes it approximate the normal CDF, so columns with wildly different units all speak the same $[0, 1]$ language. *ydist* then scores a row by Minkowski distance to “heaven”—every goal at its best—and *xdist* is the same norm over independent columns, with missing values treated pessimistically (distance 1).

On top of distance, twenty lines give active learning. Keep two pools: *best* (sorted, elite) and *rest*. Label next whichever unlabelled row is nearest *best*’s centroid and farthest from *rest*’s. After each label, if *best* has outgrown $\sqrt{1 + b + r}$, demote its worst row. The $1+$ is not decoration: it grants one row of hysteresis exactly when the pools are tiny and a one-row “best” would make the centroid pure noise. Deleting that single character shifted results on 76 of 127 benchmark datasets and cost a median point.

Budget honesty matters more than either. *acquire* takes a cap, and the evaluation harness passes Stop—Check—the labels reserved for checking the model’s suggestions are counted against the same budget as the labels that built it. Instrumenting that honestly *raised* the observed median (84 to 87): verification labels, spent on the model’s top picks, out-earn acquisition labels.

```

105 #-- distance -----
def norm(col, v):
    z = max(-3, min(3, (v - col[1]) / (1e-32 + sd(col))))
    return 1 / (1 + exp(-1.7 * z))

def mid(col):
    return max(col, key=col.get) if type(col) is Sym else col[1]

110 def mids(tbl): # centroid; only ever read over x columns
    return {at: mid(tbl.cols[at]) for at in tbl.x}

def ydist(tbl, row):
115     return (sum(abs(norm(tbl.cols[at], row[at]) - w) ** the.P
                    for at, w in tbl.y.items()) / len(tbl.y))**(1/the.P)

def _dist(col, a, b):
    if a == "?" or b == "?": return 1
    return (a != b if type(col) is Sym
            else abs(norm(col, a) - norm(col, b)))

120 def xdist(tbl, row, m):
    return (sum(_dist(tbl.cols[at], row[at], m[at]) ** the.P
                for at in tbl.x) / len(tbl.x)) ** (1 / the.P)

125 def ymu(tbl, rows):
    return sum(ydist(tbl, r) for r in rows) / len(rows)

130 def ymids(tbl, rows):
    return [sum(r[at] for r in rows) / len(rows) for at in tbl.y]

#-- acquire -----
135 def pop(tbl, best, rest, todo):
    b, r = mids(best), mids(rest)
    todo.sort(key=lambda z: xdist(tbl, z, r) - xdist(tbl, z, b))
    return todo.pop()

140 def label(tbl, best, rest, row): # keep best pool near sqrt
    addRow(best, row)
    best.rows.sort(key=lambda r: ydist(tbl, r))
    b, r = len(best.rows), len(rest.rows)
    if b > sqrt(1 + b + r): addRow(rest, addRow(best, inc=-1))

145 def acquire(tbl, cap=None):
    best, rest = clone(tbl), clone(tbl)
    todo = random.sample(tbl.rows, len(tbl.rows))[:the.Few]
    for _ in range(the.Start): label(tbl, best, rest, todo.pop())
    cap = cap or the.Stop
    while todo and len(best.rows) + len(rest.rows) < cap:
        label(tbl, best, rest, pop(tbl, best, rest, todo))
    return best.rows + rest.rows

```

Listing 3. Distance and acquisition: normalize, score against heaven, then label only the informative rows.

Bayes in three functions: `like` is a smoothed frequency (Syms) or a Gaussian pdf (Nums); `likes` multiplies one row's evidence in log space, with an m -estimate on each column and a k -smoothed prior on class size; `liked` returns the most likely of several tables. Because Num and Sym columns are already fitted summaries, there is no separate training phase: add fits, `like` reads. A classifier is just a dict of per-class clones of the table, filled by `addRow`.

`confuse` turns (got, want) pairs into per-class counts and then accuracy, recall (pd), false alarm (pf) and precision. Two details: entries are created lazily, on first sight of a class, which is safe only because true negatives are derived at the end ($tn = n - tp - fn - fp$) rather than counted per pair; and classes never predicted and never wanted simply do not appear, which is correct—they would be all- tn rows saying nothing.

Reporting *both* pd and pf is a methodological point, not a convenience. In our text-mining replication work, a recommended preprocessing step looked fine on recall alone and was exposed as counterproductive only when false alarm was measured beside it. Cheap tools make such re-checks cheap; this scorer is fourteen lines, and it grades every classifier comparison in this paper (Listing 7) on the same splits.

```

155 #-- bayes -----
def like(col, v, prior=0): # P(v | col)
    if type(col) is Sym:
        return ((col.get(v, 0) + the.m * prior)
                / (size(col) + the.m + 1e-32))
    s = sd(col) + 1e-32
    return exp(-(v-col[1])**2 / (2*s*s)) / sqrt(2*pi*s*s)

def likes(tbl, row, nall, nh): # log P(tbl | row), unscaled
    prior = (len(tbl.rows) + the.k) / (nall + the.k * nh)
    return log(prior) + sum(
        log(1e-32 + like(tbl.cols[at], v, prior))
        for at in tbl.x if (v := row[at]) != "?")

def liked(tbls, row): # most likely of several tables
    n = sum(len(t.rows) for t in tbls.values())
    return max(tbls, key=lambda k: likes(tbls[k], row, n, len(tbls)))

def confuse(pairs): # (got, want)s --> per-class scores
    out = {}
    for got, want in pairs:
        for x in [got, want]:
            out[x] = out.get(x) or o(1=x, tp=0, fp=0, fn=0)
            if got == want: out[want].tp += 1
            else: out[want].fn += 1; out[got].fp += 1
    for c in out.values():
        c.tn = len(pairs) - c.tp - c.fn - c.fp
        c.acc = (c.tp + c.tn) / len(pairs)
        c.pd = c.tp / (c.tp + c.fn + 1e-32)
        c.pf = c.fp / (c.fp + c.tn + 1e-32)
        c.prec = c.tp / (c.tp + c.fp + 1e-32)
    return out

```

Listing 4. Naive Bayes and the confusion-matrix scorer.

Trees are recursive discretization. `cutNum` sweeps sorted (x,y) pairs, sliding each value from the right summary to the left with one add each way and yielding a candidate cut only where the x value actually changes. `cutSym` tries each symbol one-vs-rest with fresh accumulators. (We considered the classic Fisher trick of ordering symbols by mean y and cutting prefixes; for nominal attributes one-vs-rest reads better as a rule, and at active-learning sizes—tens of labelled rows—the asymptotic saving is irrelevant.) Splits are scored by size-weighted expected diversity: standard deviation for numbers, entropy for symbols.

That accumulator choice is the whole classification story. The tree peeks at the type of its first y value: numeric y (distance to heaven) makes regression trees; symbolic y (a class column) makes classification trees. One line changes; the substrate does not. Missing values route by imputing the column's middle—mean for numbers, mode for symbols; the symbolic half of that symmetry was a late fix, and matters on datasets with missing nominals.

One tuning note. `Leaf=4` is tuned for the active learner, which grows trees from ~45 labelled rows. Grown passively from hundreds of rows, such trees overfit: on the diabetes data, accuracy rose from 0.69 to 0.75 simply by scaling the leaf minimum to $\sqrt{n}$. Small data and big data want different regularization, and one knob expresses both.

```

190 #-- tree -----
# Node = [edge, n, ymu, ymids, go, kid, kid]
def xpect(a, b): # sizes are >= the.Leaf, so no zero guard
    return ((div(a)*size(a) + div(b)*size(b))/(size(a) + size(b)))

def cutNum(xy, acc): # (left, right, x) per value boundary
    xy.sort()
    here, there = acc(), adds((y for _, y in xy), acc())
    for i, (x, y) in enumerate(xy[:-1]):
        here, there = add(here, y), add(there, y, -1)
        if x != xy[i+1][0]: yield here, there, x

200 def cutSym(xy, acc): # (in, out, sym), one per symbol
    for v in sorted({x for x, _ in xy}):
        yield adds((y for x, y in xy if x == v), acc()),
            adds((y for x, y in xy if x != v), acc()), v

205 def cut(tbl, rows, ys, acc): # best (col, val) split
    best = (le30, None, None)
    for at in tbl.x:
        xy = [(x, y) for r, y in zip(rows, ys) if (x := r[at]) != "?"]
        what = cutSym if type(tbl.cols[at]) is Sym else cutNum
        for here, there, v in what(xy, acc):
            if the.Leaf <= size(here) <= len(xy) - the.Leaf:
                if (s := xpect(here, there)) < best[0]:
                    best = (s, at, v)
215 if best[1] is not None: return best[1:]

def routing(tbl, at, v):
    s, c = tbl.names[at], tbl.cols[at]
    if type(c) is not Sym:
        return (f"(s) <= {round(v,2)}", f"(s) > {round(v,2)}",
                lambda r: (c[1] if r[at] == "?" else r[at]) <= v)
    return (f"(s) = {v}", f"(s) != {v}",
            lambda r: (mid(c) if r[at] == "?" else r[at]) == v)

225 def tree(tbl, rows, edge="", y=None):
    y = y or (lambda r: ydist(tbl, r))
    ys = [y(r) for r in rows]
    acc = Sym if isinstance(ys[0], str) else Num
    node = [edge, len(rows), mid(adds(ys, acc)), ymids(tbl, rows)]
    if (len(rows) > the.Leaf and (best := cut(tbl, rows, ys, acc))):
        e1, e2, go = routing(tbl, *best)
        yes, no = [], []
        for r in rows:
            (yes if go(r) else no).append(r)
235 if yes and no:
            node += [go, tree(tbl, yes, e1, y), tree(tbl, no, e2, y)]
        return node

def kids(n): return n[5:]

240 def leaf(tr, row):
    while kids(tr): tr = tr[4] if tr[4](row) else tr[6]
    return tr

```

Listing 5. Trees: recursive discretization, one accumulator swap away from classification.

Reporting walks the tree, printing each leaf's distance-to-heaven, size, and per-goal centroids, with +/- marking the best and worst leaves. The display is the model: a manager reading it is reading the actual decision boundary, not a post-hoc approximation of it, which is the property that lets tiny trees stand in for SHAP- or LIME-style explanation tools.

The stats section holds the three checks used everywhere in this paper. Cliff's delta asks whether two samples' rank imbalance exceeds 0.197 (a "small" effect); the Kolmogorov–Smirnov statistic asks whether their distributions diverge beyond the 95% line ($1.36\sqrt{(n+m)/nm}$); Cohen's d asks whether the mean gap exceeds 0.35 pooled standard deviations, with an optional absolute floor (eps) for scales, like accuracy, where tiny differences are meaningless regardless of variance. `same` declares two result distributions indistinguishable only when all three agree.

This conservatism is deliberate. Pre-registering an effect-size threshold and aggregating over twenty or more repeats rejects trivially small differences without running per-cell hypothesis tests across dozens of comparisons—the multiple-comparison correction problem never arises because the per-cell tests are never run. The whole apparatus is 26 lines, which is the reason it actually gets used: every claim of the form “X beats Y” made by this toolkit's own demos passes through same first.

```

250 #-- report -----
def leafs(tr):
    return [x for k in kids(tr) for x in leafs(k)] or [tr]

def show(tbl, tr):
    ls = sorted(leafs(tr), key=lambda z: z[2])
    print(" d2h  n" + " ".join(f"{tbl.names[at]:>7}"
                                for at in tbl.y))

    def walk(z, pre=None):
        m = "+" if z is ls[0] else "-" if z is ls[-1] else " "
        v = z[2] if type(z[2]) is str else round(100 * z[2])
        print(f"{m} {v:>3} {z[1]:>3}"
              + " ".join(f"round(v):>7" for v in z[3])
              + " (pre or " + z[0].rstrip()
              + " for k in kids(z): walk(k, " + " if pre is None else pre+ " | " )
        walk(tr)

255 #-- stats -----
def cohen(xs, ys, d=0.35, eps=0): # gap vs pooled sd, floored
    a, b = adds(xs), adds(ys)
    pool = ((a[0]-1)*sd(a)**2 + (b[0]-1)*sd(b)**2)/(a[0]+b[0]-2)
    return abs(a[1] - b[1]) <= max(eps, d * sqrt(pool))

270 def cliffs(xs, ys, d=0.197): # sorted in. rank imbalance ok?
    gt = lt = j = k = 0
    for x in xs:
        while j < len(ys) and ys[j] < x: j += 1; k = j
        while k < len(ys) and ys[k] <= x: k += 1
        gt += j; lt += len(ys) - k
    return abs(gt - lt) / (len(xs) * len(ys)) <= d

275 def ks(xs, ys, a=1.36): # sorted in. 95% kolmogorov-smirnov
    n, m, i, j, d = len(xs), len(ys), 0, 0, 0
    while i < n and j < m:
        v = min(xs[i], ys[j])
        while i < n and xs[i] <= v: i += 1
        while j < m and ys[j] <= v: j += 1
        d = max(d, abs(i/n - j/m))
    return d <= a * sqrt((n + m) / (n * m))

280 def same(xs, ys, eps=0): # indistinguishable, by all three
    xs, ys = sorted(xs), sorted(ys)
    return (cliffs(xs,ys) and ks(xs,ys) and cohen(xs,ys,eps=eps))

```

Listing 6. Reports and statistics: show the tree; ask if two treatments really differ.

wins converts distance-to-heaven into the complement of normalized regret: 100 means “found the best row in the table”, 0 means “no better than the average row”, negative means worse than that. Normalizing this way lets one number compare treatments across a hundred datasets whose raw goal scales differ by orders of magnitude.

holdout is the evaluation loop: shuffle; give the active learner half the rows (capped at Few) minus the Check labels reserved for verification; grow a tree from what it labelled; then, on the untouched half, label only the tree’s top Check picks and keep the best. Twenty repeats, report the mean. Over the 120+ MOOT optimization tasks this loop scores a median win of 86–87, and the full sweep runs in about a minute of wall-clock time on a laptop (500 cpu-seconds at 10-way parallelism)—which is precisely what makes the re-checking culture of the previous page affordable.

The first demos are worked examples of the substrate: Welford matching the textbook mean and standard deviation; the entropy of aabbbc; header routing sending columns to x, y, klass or nowhere; one legal, routable cut. Note that each demo prints its evidence (μ 5.0 σ 2.138), not “ok”: the demo teaches, the output is diffable, and a wrong number stays visible even when an assert is too loose.

```

#-- tests -----
def wins(tbl):
    ys = sorted(ydist(tbl, r) for r in tbl.rows)
    lo, b4 = ys[0], sum(ys) / len(ys)
    return lambda r: max(-100, min(100,
    100 * (1 - (ydist(tbl, r) - lo) / (b4 - lo + 1e-32))))

def holdout(tbl):
    rows = random.sample(tbl.rows, len(tbl.rows))
    n = len(rows) // 2
    train, test = rows[:n]:the.Few, rows[n:]
    tr = clone(tbl, train)
    tt = tree(tr, acquire(tr, the.Stop - the.Check))
    top = sorted(test, key=lambda r: leaf(tt, r)[21]:the.Check]
    return min(top, key=lambda r: ydist(tr, r))

#-- start-up -----
def test_help():
    "Show usage, settings, demos"
    print(__doc__, "Demos:\n",
    *["f" --{k[5]:<10} {f.__doc__}
    for k, f in globals().items() if k[5] == "test_"],
    sep="\n")

def test_num():
    "Welford add matches textbook mean and sd"
    c = adds([2, 4, 4, 4, 5, 5, 7, 9])
    assert c[0] == 8 and c[1] == 5 and abs(sd(c)-2.138) < .01
    print(f"mu {c[1]} sd {round(sd(c), 3)}")

def test_sym():
    "Syms count; mid is mode; div is entropy"
    c = adds("aabbbc", Sym())
    assert c["b"]==3 and mid(c)=="b" and abs(div(c)-1.459)<.01
    print(f"mode {mid(c)} ent {round(div(c), 3)}")

def test_tbl():
    "Headers route columns to x, y, klass, or nowhere"
    t = Tbl([["Age", "job", "Skip", "Weight-"], (2, "a", 3, 80)])
    assert t.x == [0] and t.y == {3: False} and t.klass == 1
    assert 2 not in t.cols
    print(f"x {t.x} y {t.y} klass {t.klass}")

def test_cuts():
    "cut returns a legal, routable split"
    t = Tbl(csv(the.File))
    rows = t.rows[:64]; ys = [ydist(t, r) for r in rows]
    at, v = cut(t, rows, ys, Num)
    e1, e2, go = routing(t, at, v)
    yes = sum(go(r) for r in rows)
    assert 0 < yes < len(rows)
    print(f"cut: {e1} yes={yes}; {e2} no={len(rows)-yes}")

def test_wins():
    "wins grades the best row 100"
    t = Tbl(csv(the.File))
    w = wins(t)(min(t.rows, key=lambda r: ydist(t, r)))
    assert w == 100; print(f"best row wins {w}")

```

Listing 7. Evaluation (wins, holdout) and the first demos.

The larger demos. `test_tree` grows and shows a tree on the optimization data; `test_holdout` runs the twenty-repeat evaluation; `test_klass` is a classifier bake-off in which tree and bayes see the *same* thirty 50:50 splits (a paired design that removes split-to-split variance from the comparison), their pooled confusions are printed side by side, and the statistics of Listing 6 rule on the difference. On the diabetes data the verdict reads: bayes 75% vs. tree 73%, delta 2, *different*. Naive Bayes degrades gracefully with less training data; trees fragment—exactly the regime distinction the Leaf note on Listing 5 predicts. `_klass` owns the scaffolding; each classifier is a `fit` function returning a predictor, so adding a third contender is five lines.

Below them, the harness. `run` seeds the random number generator before every demo, so any demo runs alone or in any order, and restores pristine settings in a finally (one trap: a return *inside* finally would silently swallow the crash count—the return 0 must sit outside). `cli` evaluates the argument list left to right: `-Key val sets`, `-name runs`, anything else warns and continues. The exit code counts crashes, so a Makefile gates on the suite with no framework. Any function named `test_*` is a demo, a test, and a help entry all at once: conventions are load-bearing.

```

def test_tree():
    "Acquire, grow and show the File's tree"
    tbl = Tbl(csv(the.File)); lab = acquire(tbl)
    print(f"the.File n={len(tbl.rows)}")
    f" mid={round(ymin(tbl, tbl.rows))}
    f" ezs={round(ydist(tbl, lab[0]), 3)}"
    show(tbl, tree(tbl, lab))

def test_holdout():
    "Mean win over 20 train/test holdouts"
    tbl = Tbl(csv(the.File))
    win = wins(tbl)
    mu = sum(win(holdout(tbl)) for _ in range(20)) / 20
    print(f"win {round(mu)}")

def _klass(*fits): # each fit(tbl, rows, y) --> predictor(row)
    tbl = Tbl(csv(the.Klass))
    y = lambda r: r[tbl.klass]
    n = len(tbl.rows) // 2
    splits = [random.sample(tbl.rows, len(tbl.rows))
               for _ in range(the.Repeats)] # same splits, all fits
    def one(fit):
        accs, pairs = [], []
        for rows in splits:
            got = fit(tbl, rows[:n], y)
            now = [(got(r), y(r)) for r in rows[n:]]
            pairs += now
            accs += [sum(g == w for g, w in now) / len(now)]
        for c in confuse(pairs).values():
            pc = lambda v: round(100 * v)
            print(f"fit_{tbl.klass}_{c} {pc(c.acc):>3} {pc(c.pd):>3}"
                  f" {pc(c.pf):>3} {pc(c.prec):>4}"
                  f" {tbl.cols[tbl.klass].get(c.1, 0):>6} {c.1}")
        return accs
    print(f"{'rx':<10} {'acc':>3} {'pd':>3} {'pf':>3}"
          f" {'prec':>4} {'n':>6} class")
    return [one(fit) for fit in fits]

def fitTree(tbl, rows, y): # sqrt-sized leaves
    the.Leaf = int(sqrt(len(rows)))
    tt = tree(clone(tbl, rows), rows, y=y)
    return lambda r: leaf(tt, r)[2]

def fitBayes(tbl, rows, y):
    tbsl = {}
    for r in rows:
        if y(r) not in tbsl: tbsl[y(r)] = clone(tbl)
        addRow(tbsl[y(r)], r)
    return lambda r: liked(tbsl, r)

def test_klass():
    "Tree vs bayes, same splits: confusions, then same?"
    a, b = _klass(fitTree, fitBayes)
    x, z = adds(a), adds(b)
    print(f"\nfitTree {round(100*x[1])} {round(100*sd(x))}"
          f" fitBayes {round(100*z[1])} {round(100*sd(z))}"
          f" delta {round(100*abs(x[1] - z[1]))}"
          f" : {'same' if same(a, b, eps=0.01) else 'different'}")

def test_same():
    "Stats tests tell noise from signal"
    x = [random.gauss(10, 1) for _ in range(40)]
    y = [random.gauss(10, 1) for _ in range(40)]
    z = [random.gauss(11, 1) for _ in range(40)]
    assert same(x, y) and not same(x, z)
    print(f"same {same(x, y)} diff {not same(x, z)}")

def test_all():
    "Run every demo: exit code counts the crashes"
    sys.exit(sum(print(f"\n# {k[5]}") or run(f)
                  for k, f in list(globals().items())
                  if k[5] == "test_" and f is not test_all))

def run(f=None): # demos may mutate the; always clean up
    try:
        random.seed(the.Seed); (f or test_help)()
    except Exception: traceback.print_exc(); return 1
    finally:
        vars(the).update(vars(defaults))
    return 0

def cli(d, funs, args, n=0):
    while args:
        s = args.pop(0)
        if s[2] == "--" : n += run(funs.get("test_" + s[2:]))
        elif s[1:] in d : d[s[1:]] = atom(args.pop(0))
        else: print(f"unknown arg: {s}")
    sys.exit(n)

def main(): # pip entry point
    cli(vars(the), globals(), sys.argv[1:] or ["--help"])

if __name__ == "__main__": main()

```

Listing 8. The larger demos and the harness: a statistically-checked bake-off, then sixteen lines that replace a test framework.